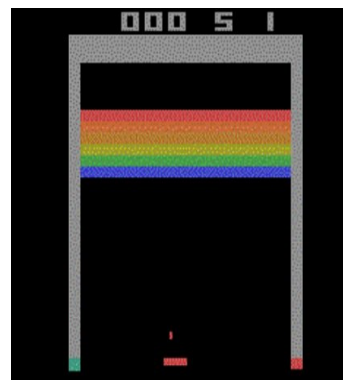


Deep Reinforcement Learning — Assignment 1

Ido Raviel & Dan Butensky

Introduction

This report documents our implementation of Deep Q-Network (DQN) applied to the Atari game Breakout. We implemented the algorithm from scratch in PyTorch, with the addition of a target network to stabilize training. The agent learns directly from raw pixel observations, using a convolutional neural network to approximate the action-value function $Q(s, a)$.



Preprocessing

We applied a standard Atari preprocessing pipeline, implemented as a stack of Gymnasium wrappers in the following order:

- **Frame Skip (MaxAndSkipObservation):** Every action selected by the agent is repeated for 4 consecutive raw ALE frames, and the resulting observations are max-pooled. This reduces the decision frequency and speeds up training.
- **Fire on Reset (FireResetEnv):** Breakout requires pressing FIRE after each reset to launch the ball. Without this, the episode stalls on a static screen. This wrapper automatically injects a FIRE action after every reset.
- **No-op Randomization (NoopResetEnv):** After FIRE, we apply 1–7 random no-op actions (each covering 4 raw frames, so up to 28 raw frames total). This randomizes the starting ball position and prevents the agent from memorizing a fixed opening sequence.
- **Image Processing (AtariPreprocessing):** Each frame is converted from RGB (210×160×3) to grayscale, resized to 84×110, then cropped to 84×84 by removing the score area at the top. Rewards are clipped to $[-1, 1]$. During training, life loss is handled without treating it as a terminal signal (see What Worked); during evaluation, a FIRE action is automatically injected to resume play after a life is lost.
- **Frame Stacking (FrameStack):** The 4 most recent processed frames are stacked into a (4, 84, 84) tensor, giving the agent a sense of motion and ball direction from a single observation.

Network Architecture

We used the 2015 Nature DQN architecture. The input is a (4, 84, 84) tensor of stacked grayscale frames, normalized to $[0, 1]$ at sample time. The network consists of three convolutional layers followed by two fully connected layers, with ReLU activations on all hidden layers. The output layer produces one Q-value per action with no activation.

Layer	Details	Output Shape
Conv1	32 filters, 8×8, stride 4	(32, 20, 20)
Conv2	64 filters, 4×4, stride 2	(64, 9, 9)
Conv3	64 filters, 3×3, stride 1	(64, 7, 7)
FC1	3136 → 512 units	512
Output	512 → n_actions	n_actions

Hyperparameters

Parameter	Value
Optimizer	RMSprop ($\alpha=0.95$, $\epsilon=0.01$)
Learning rate	0.00025
Discount factor (γ)	0.99
Batch size	32
Replay buffer capacity	1M
Replay buffer warmup	50K transitions
Epsilon start → end	1.0 → 0.1
Epsilon decay period	1.25M env steps (5M raw frames)
Evaluation epsilon	0.05
Target network update	Every 10K gradient updates
Gradient update frequency	Every 4 env steps (every 16 raw frames)
Total training steps	25M env steps (100M raw frames)
Evaluation frequency	Every 10K gradient updates
Loss function	Huber loss (smooth L1)
Held-out states for Q tracking	2K

What Didn't Work

Too Few Training Steps

In our first attempt we trained for 12.5M env steps (50M raw frames) with a shorter epsilon decay period of 250K steps. While the agent did show signs of learning — reaching scores in the 60–136 range by the end — it was clearly still improving when training terminated. The Q-values were steadily rising and the reward curve had not plateaued, indicating the agent simply ran out of time to converge. Breakout requires the agent to discover and refine a long-horizon strategy, which demands significantly more environment interaction than we initially allocated.

Reward Shaping with sqrt Normalization

We experimented with replacing the standard reward clipping with a sqrt-based normalization ($\sqrt{\text{reward} / 7}$), motivated by the idea that it would preserve the relative value of different brick rows. The brick rows have the following point values:

Row (top → bottom)	Color	Raw Score	Clipped (paper)	$\sqrt{r/7}$
1–2	Red	7	1	1.00
3–4	Orange	7	1	1.00
5–6	Yellow	4	1	0.76
7–8	Green	4	1	0.76
9–10	Aqua	1	1	0.38
11–12	Blue	1	1	0.38

We hoped this graded signal would encourage the agent to aim for higher rows and learn the tunnel strategy, where the ball breaks through to the space above the bricks and scores rapidly. In practice, the agent performed significantly worse, reaching only scores of 23–75 at the end of 25M steps, compared to 100–300 with standard clipping. We believe the rescaled, low-magnitude rewards produced weaker TD targets and slowed learning, ultimately preventing the agent from developing any consistent strategy.

What Worked

Standard Reward Clipping

Clipping all rewards to $[-1, 1]$ proved to be the more effective reward normalization strategy. Compared to the sqrt-based approach, it provided a consistent and stable training signal across all brick types. While it treats all bricks as equal regardless of their point value, this simplicity appears to benefit learning — the agent receives a clear, unambiguous reward on every hit without the weaker, fractional signals introduced by normalization.

Epsilon Decay Proportional to Total Training Steps

In the original paper, epsilon decays over approximately 2% of the total training budget. Since we trained for 25M steps rather than the paper's shorter schedule, we scaled the decay period to 1.25M steps — roughly 5% of total training — giving the agent slightly more exploration time relative to the longer run. This ensured the agent continued exploring meaningfully during the early phase of training rather than committing to a greedy policy too soon, which was especially important given the additional training time available for convergence.

Disabling Terminal on Life Loss

The original DQN paper treats each life loss as a terminal state during training, forcing the agent to assign zero future value after losing a life. We experimented with disabling this — setting `terminal_on_life_loss=False` — so the agent experiences the full game trajectory across all 5 lives without artificial episode resets. This produced noticeably better results: Q-values reached ~3.3–3.6 compared to ~2.1–2.3 with termination enabled, and the best checkpoint achieved a mean score of 290.83 over 30 games (vs. 245.93 previously). We believe this allows the agent to learn longer-horizon strategies, particularly the tunnel strategy where the ball scores many points in a single life, rather than

optimizing purely for short-term survival.

Results

All three runs were trained for 25M env steps (100M raw frames) with `terminal_on_life_loss=False`. Evaluation was performed every 10K gradient updates, consisting of 1 full episode (all 5 lives) with $\epsilon=0.05$. All three runs show consistent learning: near-zero rewards during the first ~5M steps, steady improvement between 5M–15M steps, and strongly elevated scores in the final phase (15M–25M steps) with peaks reaching 400+. The Q-value curves are smooth and monotonically increasing, reaching ~3.3–3.6 by the end of training — significantly higher than the `terminal_on_life_loss=True` runs — confirming stable learning without divergence.

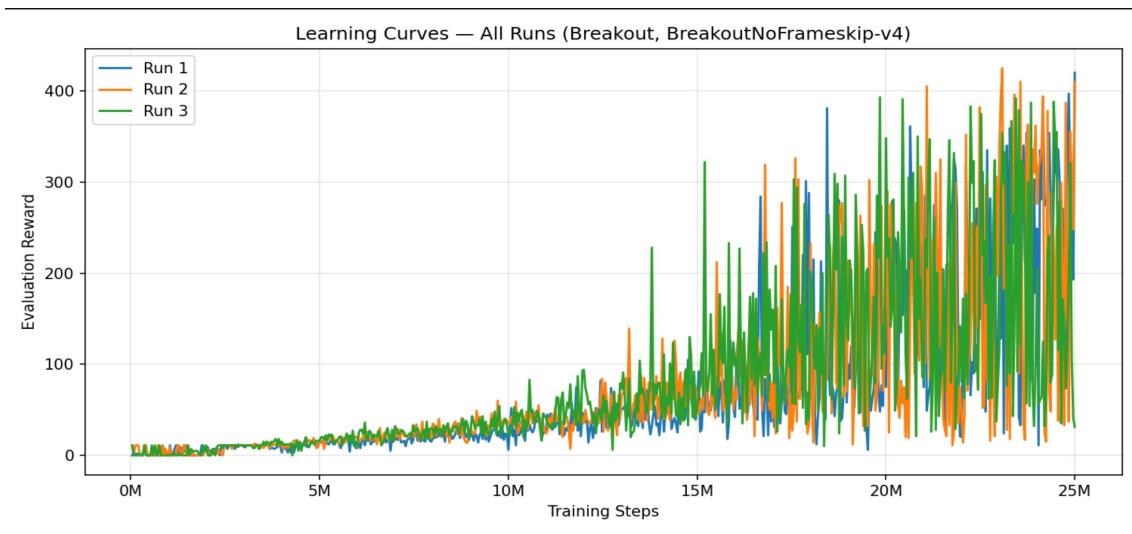


Figure 1: Evaluation reward across all 3 runs over 25M training steps.

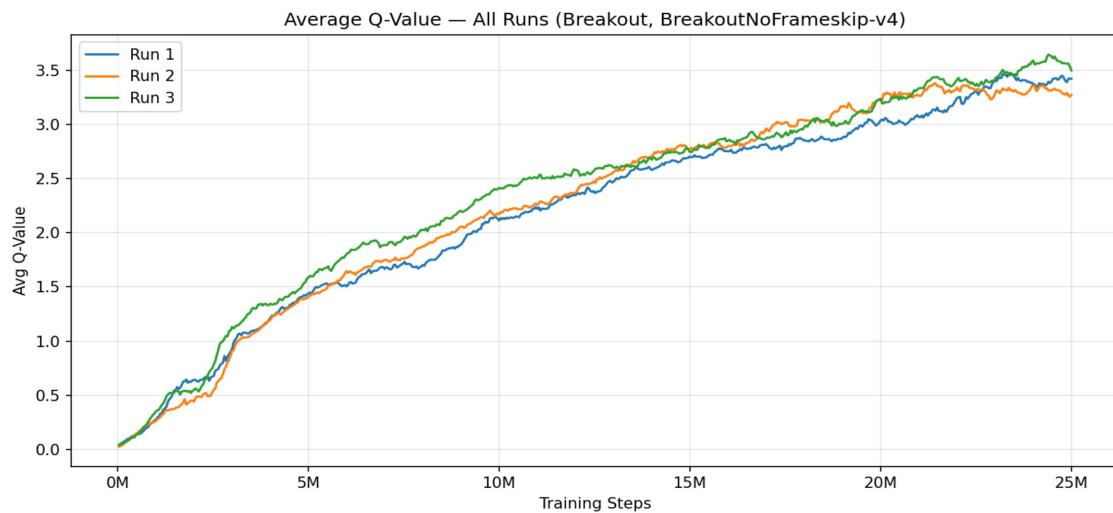


Figure 2: Average Q-value across all 3 runs over 25M training steps.

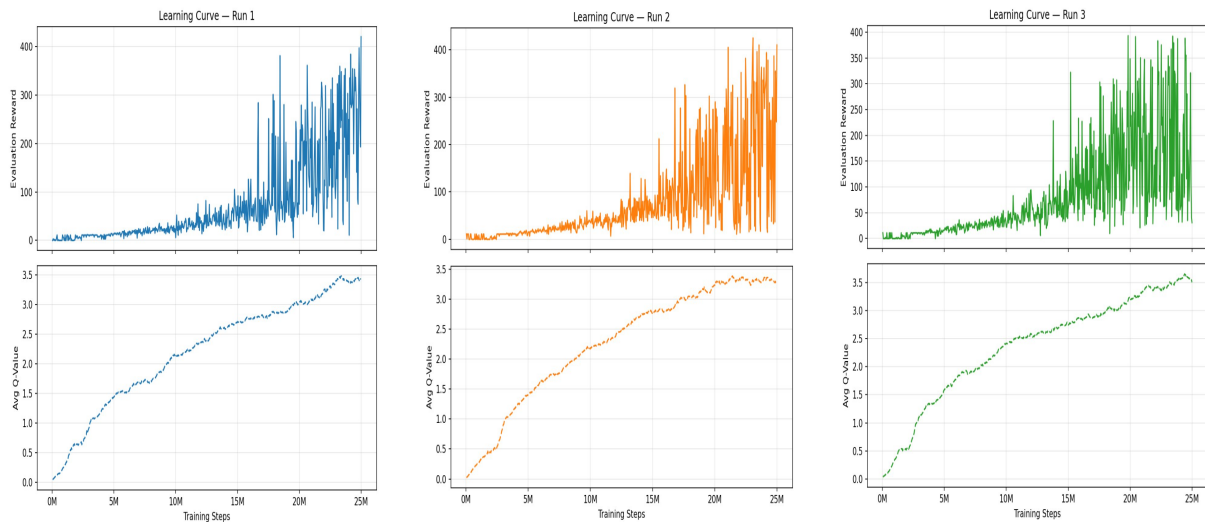


Figure 3: Individual learning curves (reward + Q-value) for Run 1, Run 2, and Run 3.

Final rewards at the last evaluation checkpoint (step 25M):

	Run 1	Run 2	Run 3	Average
Final Reward	420.0	410.0	31.0	287.0

Note: the final checkpoint reward is noisy due to single-episode evaluation. Run 3's low final value (31.0) is likely an unlucky episode — its Q-value curve shows healthy learning throughout. The best saved checkpoint (Run 2) was evaluated more rigorously — see the subsection below.

Best Checkpoint Evaluation

The best saved checkpoint (Run 2) was evaluated over 30 full games ($\epsilon=0.05$), providing a more reliable estimate of the agent's true performance. Results are summarized below.

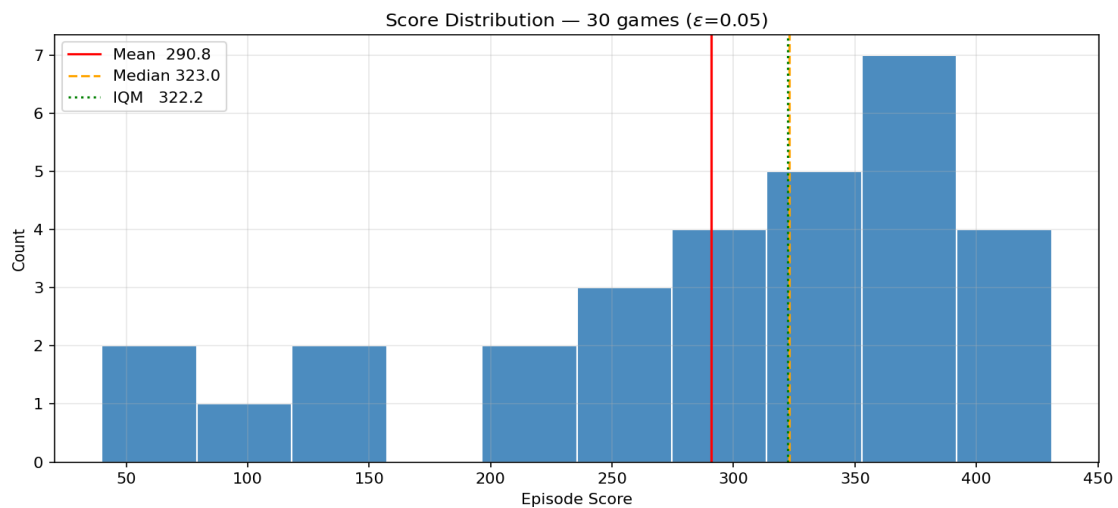
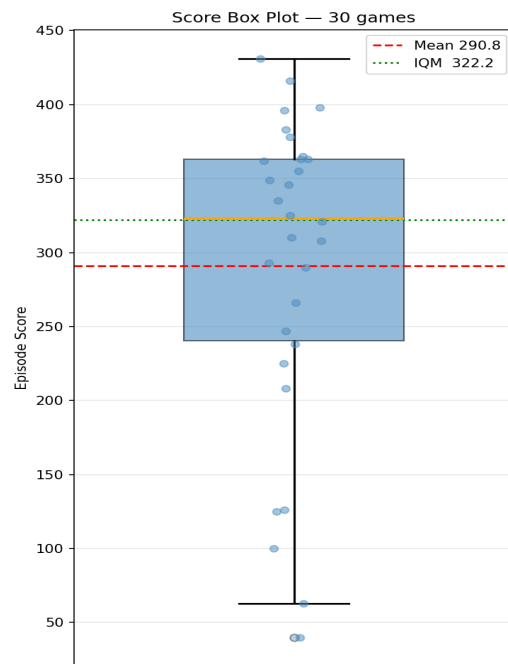


Figure 4: Score distribution over 30 evaluation episodes.



Evaluation Summary — 30 games ($\epsilon=0.05$)

Metric	Value
Mean	290.83
Std	105.29
Median	323.00
IQM	322.20
Min	40
Max	431

Figure 5: Box plot of episode scores (left) and evaluation summary statistics (right).